

2026 年 4 月 29 日 Version x.x

---

# ハッカソン ～計画書と設計書～

---

## チーム 23

24G1038: 片岡 聖  
24G1075: 地曳 賢人  
24G1131: 御代川 稜  
25G1021: 梅澤 颯太  
25G1053: 齋藤 翔太  
26G1065: 塩澤 匠生



# 目次

|       |                       |    |
|-------|-----------------------|----|
| 第 1 章 | システムの基本設計 .....       | 5  |
| 1.1   | 機能一覧 .....            | 5  |
| 1.2   | システム構成図 .....         | 5  |
| 1.3   | 操作インターフェース設計 .....    | 6  |
| 1.4   | 状態遷移図 .....           | 6  |
| 第 2 章 | システムの詳細設計 .....       | 9  |
| 2.1   | 部品一覧 .....            | 9  |
| 2.2   | ハードウェア設計 .....        | 9  |
| 2.3   | ソフトウェア設計 .....        | 10 |
| 2.3.1 | ファイル構成 .....          | 10 |
| 2.3.2 | オブジェクト設計 (クラス図) ..... | 10 |
| 2.3.3 | 関数設計 .....            | 10 |
| 2.4   | 処理フローの設計 .....        | 11 |
| 2.5   | テストの設計 .....          | 11 |
| 2.6   | AI の利用について .....      | 11 |



# 第1章 システムの基本設計

本章では、指揮者のジェスチャによる BPM 制御と、複数の端末間での高精度な演奏同期を実現するためのシステム設計について述べる。

## 1.1 機能一覧

本システムの機能分解構造（FBS）を以下に定義する。

- ・ **時刻同期・キャリブレーション機能**：演奏開始前に指揮棒と各楽器間の内部時刻を PTP-lite 方式で同期し、ネットワーク遅延に伴うオフセットを解消する機能。
- ・ **指揮動作解析機能**：6 軸加速度センサの出力から、指揮の「静」と「動」が切り替わる打点を検出し、生の BPM 値を算出する機能。
- ・ **テンポ量子化・ヒステリシス制御機能**：算出した BPM を音楽的な標準テンポにマッピングする。130 BPM（上昇時）および 125 BPM（下降時）の閾値を用いたヒステリシス処理により、境界付近での不要なテンポ変動を抑制する機能。
- ・ **予約実行型 UDP 通信機能**：確定した BPM 情報と、実行予定時刻 ( $T_{exec}$ ) を含む独自プロトコルのパケットを生成する。UDP ブロードキャストおよび冗長送出（複数回連打）によって、全機へ低遅延かつ確実に同時配信する機能。
- ・ **分散音響合成機能**：各楽器ユニットにシリアル接続された Mac（Processing）において、ADSR およびエフェクト処理を伴う独立した音源合成を行い、内蔵スピーカーから出力する機能。

## 1.2 システム構成図

ハードウェアおよびソフトウェアの構成を以下に示す。

- ・ **ハードウェア構成**：
  - 指揮棒ユニット：Arduino Uno R4 WiFi × 1 台、6 軸加速度センサ × 1 個

- 楽器ユニット：Arduino Uno R4 WiFi × 4 台
- 音響出力・処理：ノート PC (Mac) × 4 台
- 接続構成：指揮棒と楽器間は WiFi (UDP) 接続，各楽器 Arduino と Mac は USB シリアル接続.
- ソフトウェア構成：
  - 制御プログラム：C++ (Arduino 言語)
  - 通信プロトコル：独自バイナリ形式プロトコル (UDP/IP)
  - 音響合成エンジン：Processing (Sound ライブラリ)

## 1.3 操作インターフェース設計

演奏者および指揮者の操作性を考慮したインターフェース設計を以下に定める.

- 指揮棒ユニット：
  - 入力：6 軸センサによる空間ジェスチャ. また，同期開始用の物理ボタンまたは特定動作.
  - 出力：LED マトリクスによる同期完了通知および，現在の量子化テンポ状態 (120/140 等) の表示.
- 楽器ユニット：
  - 出力：LED マトリクスによる拍子 (BPM タイミング) の視覚的フィードバック.
- 音響モニタ (PC 画面)：Processing の GUI 上に，現在の BPM，ADSR の状態，音声波形，および通信パケットの受信ログをリアルタイムで表示する.

## 1.4 状態遷移図

システム全体の主要な動作状態と遷移条件を以下に定義する.

1. Initialization (初期化)：電源投入後，各ユニットが WiFi ネットワークへ接続し，センサのキャリブレーションを行う状態.
2. Sync Mode (時刻同期)：指揮者の合図により開始. マスターと各スレーブ間で時刻同期パケットを交換し，内部時計のズレ (Offset) を補正する. 完了後，全機

が Ready へ移行.

3. Ready (演奏待機): 全機の LED が点灯または点滅し, 演奏開始準備が整った状態. 指揮の第一打 (振り下ろし) を待機する.
4. Playing (演奏中): 指揮の BPM をリアルタイムに解析・配信し, 譜面を進行させる状態.
  - 120 BPM で演奏中, 計測値が 130 BPM を超えたら 140 BPM へ遷移パケットを送信.
  - 140 BPM で演奏中, 計測値が 125 BPM を下回ったら 120 BPM へ遷移パケットを送信.
5. Sync-Loss (同期維持): 一時的な通信途絶時, 直前の BPM と自身の内部時計に基づき, 自立的に演奏を継続するフェイルセーフ状態.
6. Stop (停止): 演奏終了動作の検知により音出力を停止し, Ready 状態へ戻る状態.





## 第 2 章 システムの詳細設計

本章では，基本設計で定義した各機能を実装するための具体的なハードウェア構成，ソフトウェア構造，および処理アルゴリズムについて詳述する．

### 2.1 部品一覧

本システムを構成する製品分解構造（PBS）の主要部品は以下の通りである．

- **指揮棒ユニット：**

- Arduino Uno R4 WiFi：メイン制御および WiFi 通信．
- 6 軸加速度・ジャイロセンサ（BMI270 等）：指揮動作の検知．
- 物理ボタン：時刻同期キャリブレーションのトリガー．

- **楽器ユニット（計 4 セット）：**

- Arduino Uno R4 WiFi：通信および演奏シーケンス制御．
- LED マトリクス（R4 内蔵）：メトロノーム視覚フィードバック．

- **音源ユニット（計 4 セット）：**

- ノート PC (Mac)：Processing による音響合成．
- USB ケーブル：Arduino-PC 間のシリアル通信用．

### 2.2 ハードウェア設計

各ユニットの接続構成は以下の通りである．

- 指揮棒側のセンサは I2C バスを介して Arduino に接続し，高頻度（100Hz 以上）でサンプリングを行う．
- 全 Arduino は共通の SSID を持つ WiFi アクセスポイントに接続し，固定 IP アドレスを割り当てることでアドレッシングの高速化を図る．
- 楽器 Arduino から Mac へのデータ伝送は，ボーレート 115,200bps 以上の高速シリ

アル通信を使用し、転送遅延を最小化する。

## 2.3 ソフトウェア設計

### 2.3.1 ファイル構成

- Conductor\_Main.ino：指揮棒側主処理（センサ解析，BPM 決定，UDP 送信）。
- Instrument\_Main.ino：楽器側主処理（UDP 受信，スケジューラ，シリアル送信）。
- SyncManager.h/cpp：PTP-lite に基づく時刻同期ロジック。
- ProtocolDefinition.h：独自バイナリプロトコルの構造体定義。
- SoundEngine.pde：Processing 用音響合成プログラム（Mac 側）。

### 2.3.2 オブジェクト設計 (クラス図)

- TimeSyncManager クラス：マスター・スレーブ間のオフセット計算と時刻補正を行う。
- TempoController クラス：130/125 BPM の閾値を用いたヒステリシス制御をカプセル化する。
- EventQueue クラス：予約時刻付きの演奏命令を受信時刻順に保持し，指定時刻に実行トリガーを出す。

### 2.3.3 関数設計

- float calculateBPM()：加速度のゼロクロス間隔から BPM を算出する。
- int quantizeTempo(float rawBPM)：ヒステリシス論理に基づき，標準テンポを決定する。
- void sendRedundantPacket(Packet p)：同一 UDP パケットを短間隔で 3 回送信する。
- void checkAndExecute()：楽器側メインループ内で呼び出され，現在時刻が予約時刻に達した命令を実行する。

## 2.4 処理フローの設計

主要な処理アルゴリズムのフローを以下に定義する.

1. **時刻同期フロー**：指揮棒が SYNC\_REQ をブロードキャスト → 各楽器が受信時刻と共に ACK 返信 → 指揮棒が往復遅延を計算し補正値を返信 → 各楽器が内部時計 (millis()) にオフセットを適用.
2. **テンポ制御フロー (ヒステリシス)**：
  - 現在 120 BPM の場合：新計測値が 130 を超えたら 140 へ遷移予約.
  - 現在 140 BPM の場合：新計測値が 125 を下回ったら 120 へ遷移予約.
3. **予約実行フロー**：指揮棒が「現在時刻 + 2000ms」を実行時刻としたパケットを送信 → 楽器側は受信後キューに保存 → if (myTime >= T\_exec) となった瞬間にシリアル経由で Mac へ発音指示を送出.

## 2.5 AI の利用について

文章の校正, 関数名クラス名などの提案などに利用した.